

Java Autoboxing and Unboxing

Java provides **Autoboxing** and **Unboxing** to automatically convert between **primitive data types** and their corresponding **wrapper classes**.

Primitive Type	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Autoboxing

Definition:

Autoboxing is the automatic conversion of a primitive type into its corresponding wrapper object by the Java compiler.

```
public class AutoBoxingDemo {
    public static void main(String[] args) {

        int a = 100;

        // Autoboxing
        Integer obj = a; //Integer obj = Integer.valueOf(a);

        System.out.println("Primitive: " + a);
        System.out.println("Wrapper Object: " + obj);
    }
}
```

Primitive: 100
Wrapper Object: 100

Autoboxing for All Wrapper Classes

Primitive	Wrapper Class	Autoboxing
byte	Byte	Byte b = x;
short	Short	Short s = x;
int	Integer	Integer i = x;
long	Long	Long l = x;
float	Float	Float f = x;
double	Double	Double d = x;
char	Character	Character c = x;
boolean	Boolean	Boolean b = x;

```
public class AutoBoxingAll {
    public static void main(String[] args) {

        byte b = 10;
        Byte byteObj = b;          // Byte.valueOf(b)

        short s = 20;
        Short shortObj = s;        // Short.valueOf(s)

        int i = 100;
        Integer intObj = i;        // Integer.valueOf(i)

        long l = 1000L;
        Long longObj = l;          // Long.valueOf(l)

        float f = 12.5f;
        Float floatObj = f;        // Float.valueOf(f)

        double d = 99.99;
        Double doubleObj = d;      // Double.valueOf(d)

        char ch = 'A';
        Character charObj = ch;    // Character.valueOf(ch)

        boolean flag = true;
        Boolean boolObj = flag;    // Boolean.valueOf(flag)

        System.out.println("Byte      : " + byteObj);
        System.out.println("Short     : " + shortObj);
        System.out.println("Integer   : " + intObj);
        System.out.println("Long      : " + longObj);
        System.out.println("Float     : " + floatObj);
        System.out.println("Double    : " + doubleObj);
        System.out.println("Character : " + charObj);
        System.out.println("Boolean   : " + boolObj);
    }
}
Byte      : 10
Short     : 20
Integer   : 100
Long      : 1000
Float     : 12.5
Double    : 99.99
Character : A
Boolean   : true
```

Array vs ArrayList

Array	ArrayList
Fixed size	Dynamic (Resizable)
Can store primitives and objects	Stores only objects (use wrapper classes for primitives)
Length cannot change	Size can increase or decrease
Example: <code>int[] a = {1,2,3};</code>	Example: <code>ArrayList<Integer> list = new ArrayList<>();</code>

Autoboxing in ArrayList

```
import java.util.ArrayList;
public class AutoBoxingList {
    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10); // int → Integer Autoboxing means list.add(Integer.valueOf(10));
        list.add(20);
        list.add(30);
        System.out.println(list);
    }
}
[10, 20, 30]
```

Unboxing

Definition:

Unboxing is the automatic conversion of a wrapper object into its corresponding primitive type.

```
public class UnBoxingDemo {
    public static void main(String[] args) {
        Integer obj = 200;
        // Unboxing
        int a = obj; //int a = obj.intValue();

        System.out.println("Wrapper Object: " + obj);
        System.out.println("Primitive: " + a);
    }
}
Wrapper Object: 200
Primitive: 200
```

Unboxing for All Wrapper Classes

Wrapper Class	Primitive Type	Unboxing	Manual Equivalent
Byte	byte	byte b = byteObj;	byteObj.byteValue()
Short	short	short s = shortObj;	shortObj.shortValue()
Integer	int	int i = intObj;	intObj.intValue()
Long	long	long l = longObj;	longObj.longValue()
Float	float	float f = floatObj;	floatObj.floatValue()
Double	double	double d = doubleObj;	doubleObj.doubleValue()
Character	char	char ch = charObj;	charObj.charValue()
Boolean	boolean	boolean flag = boolObj;	boolObj.booleanValue()

```

public class UnBoxingAll {
    public static void main(String[] args) {

        Byte byteObj = 10;
        byte b = byteObj;           // byteObj.byteValue()

        Short shortObj = 20;
        short s = shortObj;         // shortObj.shortValue()

        Integer intObj = 100;
        int i = intObj;             // intObj.intValue()

        Long longObj = 1000L;
        long l = longObj;           // longObj.longValue()

        Float floatObj = 12.5f;
        float f = floatObj;         // floatObj.floatValue()

        Double doubleObj = 99.99;
        double d = doubleObj;       // doubleObj.doubleValue()

        Character charObj = 'A';
        char ch = charObj;          // charObj.charValue()

        Boolean boolObj = true;
        boolean flag = boolObj;     // boolObj.booleanValue()

        System.out.println("Byte    : " + b);
        System.out.println("Short   : " + s);
        System.out.println("Integer : " + i);
        System.out.println("Long    : " + l);
        System.out.println("Float   : " + f);
        System.out.println("Double  : " + d);
        System.out.println("Character : " + ch);
        System.out.println("Boolean : " + flag);
    }
}

```

```
Byte    : 10
Short   : 20
Integer : 100
Long    : 1000
Float   : 12.5
Double  : 99.99
Character : A
Boolean : true
```

Unboxing from ArrayList

```
import java.util.ArrayList;

public class UnBoxingList {
    public static void main(String[] args) {

        ArrayList<Integer> list = new ArrayList<>();

        list.add(100);
        list.add(200);

        int x = list.get(0); // Integer → int

        System.out.println(x);
    }
}
100
```

Autoboxing and Unboxing Together

```
public class AutoUnboxDemo {
    public static void main(String[] args) {

        int x = 10;

        // Autoboxing
        Integer obj = x;

        // Unboxing
        int y = obj;

        System.out.println("x = " + x);
        System.out.println("obj = " + obj);
        System.out.println("y = " + y);
    }
}
x = 10
obj = 10
y = 10
```

Arithmetic Using Wrapper Objects

```
public class WrapperArithmetic {  
    public static void main(String[] args) {  
  
        Integer a = 50;  
        Integer b = 25;  
  
        Integer sum = a + b;  
  
        System.out.println(sum);  
    }  
}
```

75

What happens internally?

1. a is unboxed to int.
2. b is unboxed to int.
3. Addition is performed.
4. The result is autoboxed into an Integer.

NullPointerException During Unboxing

```
public class NullUnboxing {  
    public static void main(String[] args) {  
        Integer num = null;  
        int x = num;    // Runtime Exception  
        System.out.println(x);  
    }  
}
```

Exception in thread "main"

java.lang.NullPointerException

Reason: null cannot be converted to a primitive value